

```
import requests
import json
from datetime import datetime, timedelta
from dateutil import parser

laptops = [
    {
        "id": "PG026RXY",
        "hostname": "roshni-ThinkPad-E14",
        "bucket": "aw-watcher-window_roshni-ThinkPad-E14",
        "host": "localhost"
    },
]

category_map = {
    "browsing": ["chrome", "firefox", "edge", "brave"],
    "programming": ["code", "vscode", "sublime", "pycharm", "terminal",
"gnome-terminal"],
    "learning": ["notion", "obsidian", "anki", "pdf", "evince", "document
viewer", "libreoffice", "writer", "reader"],
    "communication": ["slack", "discord", "teams", "zoom", "skype",
"meet"],
    "entertainment": ["spotify", "vlc", "mpv", "netflix", "youtube", "prime
video", "media player"],
    "other": []
}

def categorize(app_name):
    if not app_name:
        return "other"
    app_name = app_name.lower()
    for category, apps in category_map.items():
        if any(app in app_name for app in apps):
            return category
    return "other"

def format_duration(seconds):
    if not isinstance(seconds, (int, float)):
        return "0 sec"
```

```

seconds = int(seconds) # Force integer seconds
minutes = seconds // 60
if minutes >= 60:
    hrs = minutes // 60
    mins = minutes % 60
    return f"{int(hrs)} hr {int(mins)} min" if mins else f"{int(hrs)}
hr"
elif minutes > 0:
    return f"{int(minutes)} min"
else:
    return f"{int(seconds)} sec"

# Function to calculate off times
def get_off_times(events, start_time, end_time, threshold_minutes=15):
    from datetime import datetime as dt

    normalized = []
    for e in events:
        try:
            t = e["time"]
            parsed_time = parser.isoparse(t)
            if parsed_time.tzinfo is not None:
                parsed_time = parsed_time.astimezone().replace(tzinfo=None)
            normalized.append(parsed_time)
        except Exception as err:
            print(f"Skipping bad time '{e.get('time')}': {err}")

    normalized.sort()
    off_times = []

    # If no events, whole period is off
    if not normalized:
        total_off = (end_time - start_time).total_seconds()
        off_times.append({
            "start": start_time.strftime("%H:%M"),
            "end": end_time.strftime("%H:%M"),
            "duration_sec": total_off
        })
    return off_times

```

```

# Gap before first event
first_gap = (normalized[0] - start_time).total_seconds() / 60
if first_gap >= threshold_minutes:
    off_times.append({
        "start": start_time.strftime("%H:%M"),
        "end": normalized[0].strftime("%H:%M"),
        "duration_sec": first_gap * 60
    })

# Gaps between events
for prev, curr in zip(normalized[:-1], normalized[1:]):
    diff_min = (curr - prev).total_seconds() / 60
    if diff_min >= threshold_minutes:
        off_times.append({
            "start": prev.strftime("%H:%M"),
            "end": curr.strftime("%H:%M"),
            "duration_sec": diff_min * 60
        })

# Gap after last event
last_gap = (end_time - normalized[-1]).total_seconds() / 60
if last_gap >= threshold_minutes:
    off_times.append({
        "start": normalized[-1].strftime("%H:%M"),
        "end": end_time.strftime("%H:%M"),
        "duration_sec": last_gap * 60
    })

return off_times

# Time range
now = datetime.now()
start_time = now - timedelta(hours=24)
end_time = now

print(f"Generated on: {now.strftime('%Y-%m-%d %H:%M')}")
print("Start:", start_time.strftime("%Y-%m-%dT%H:%M:%SZ"))
print("End:", end_time.strftime("%Y-%m-%dT%H:%M:%SZ"))

```

```

result = {}
category_summary = {}

for laptop in laptops:
    category_summary[laptop['id']] = {
        "usage": {cat: "0 sec" for cat in category_map.keys()},
        "afk": "0 sec"
    }

    # --- WINDOW EVENTS ---
    events_url =
f"http://{laptop['host']}:5600/api/0/buckets/{laptop['bucket']}/events?sta
rt={start_time.strftime('%Y-%m-%dT%H:%M:%SZ')}&end={end_time.strftime('%Y-
%m-%dT%H:%M:%SZ')}"
    try:
        res = requests.get(events_url, timeout=10)
        if res.status_code != 200:
            print(f"Error fetching window events for {laptop['id']}:
{res.status_code}")
            continue

        events = res.json()
        if not isinstance(events, list):
            print(f"Invalid events format for {laptop['id']}")
            continue

        formatted = []
        usage_totals = {category: 0 for category in category_map.keys()}

        for e in events:
            try:
                ts =
parser.isoparse(e['timestamp']).astimezone().replace(tzinfo=None)
                app = e['data'].get('app', '')
                category = categorize(app)
                duration_sec = int(e['duration'])
                usage_totals[category] += duration_sec
                formatted.append({
                    "time": ts.isoformat(),
                    "app": app,

```

```

        "title": e['data'].get('title', ''),
        "duration": format_duration(duration_sec),
        "category": category
    })
except Exception as e:
    print(f"Error processing event: {e}")

result[laptop['id']] = formatted
category_summary[laptop['id']]["usage"] = {cat:
format_duration(sec) for cat, sec in usage_totals.items()}
category_summary[laptop['id']]["total_usage"] =
format_duration(sum(usage_totals.values()))

except Exception as e:
    print(f"Window events error for {laptop['id']}: {str(e)}")
    result[laptop['id']] = [{"error": str(e)}]

# --- AFK EVENTS ---
afk_bucket = f"aw-watcher-afk_{laptop['hostname']}"
afk_url =
f"http://{laptop['host']}:5600/api/0/buckets/{afk_bucket}/events?start={st
art_time.strftime('%Y-%m-%dT%H:%M:%SZ')}&end={end_time.strftime('%Y-%m-%dT
%H:%M:%SZ')}"
try:
    afk_res = requests.get(afk_url, timeout=10)
    if afk_res.status_code == 200:
        afk_events = afk_res.json()
        if isinstance(afk_events, list):
            total_afk = sum(int(a.get("duration",0)) for a in
afk_events)
            category_summary[laptop['id']]["afk"] =
format_duration(total_afk)
            for afk in afk_events:
                ts =
parser.isoparse(afk['timestamp']).astimezone().replace(tzinfo=None)
                result[laptop['id']].append({
                    "time": ts.isoformat(),
                    "app": "-",
                    "title": "User Idle",

```

```

        "duration":
format_duration(int(afk.get("duration",0))),
        "category": "afk"
    })
except Exception as e:
    print(f"AFK events error for {laptop['id']}: {e}")

# --- LAPTOP OFF TIMES ---
off_periods = get_off_times(result[laptop['id']], start_time, end_time,
threshold_minutes=15)
category_summary[laptop['id']]['off_times'] = [{
    "start": o["start"],
    "end": o["end"],
    "duration": format_duration(o["duration_sec"])
} for o in off_periods]

total_off_seconds = sum(o["duration_sec"] for o in off_periods)
category_summary[laptop['id']]['total_off_time'] =
format_duration(total_off_seconds)

# Save outputs
with open("activity_cache.json", "w") as f:
    json.dump(result, f, indent=2, default=str)

with open("category_summary.json", "w") as f:
    json.dump(category_summary, f, indent=2, default=str)

# --- SEND TO GOOGLE SCRIPT ---
def send_to_google_script(laptop_id, summary_data, activity_data):
    APPS_SCRIPT_URL =
"https://script.google.com/macros/s/AKfycbyNAvN4kwWEkeRQFVKXtZaUI8ijRakGxW
JQB-XgabrPtrZosS8XlGhZauQv4RvUsMPFpg/exec"
    summary_text = "\n".join([f"{cat}: {time}" for cat, time in
summary_data['usage'].items()])
    summary_text += f"\nAFK Time: {summary_data['afk']}"

    if "off_times" in summary_data:
        summary_text += "\nLaptop Off Periods:"
        for off in summary_data["off_times"]:

```

```

        summary_text += f"\n - {off['start']} to {off['end']}
({off['duration']})"

payload = {
    "laptopId": laptop_id,
    "summary": summary_text,
    "activity": activity_data,
    "afk": summary_data.get("afk", "0 min"),
    "total_usage": summary_data.get("total_usage", "0 min"),
    "off_times": summary_data.get("total_off_time", "0 min"),
    "type": "ActivityPdf",
    "timeRange": "Last 24 hours"
}

try:
    response = requests.post(APPS_SCRIPT_URL, json=payload, timeout=60)
    if response.status_code == 200:
        try:
            js = response.json()
            if js.get('status')=="success":
                print(f"✅ PDF generated: {js.get('url')}")
            else:
                print(f"❌ Script error: {js.get('message')}")
        except:
            print(f"⚠️ Unexpected response format")
    else:
        print(f"❌ HTTP Error {response.status_code}: {response.text}")
except Exception as e:
    print(f"❌ Connection failed: {e}")

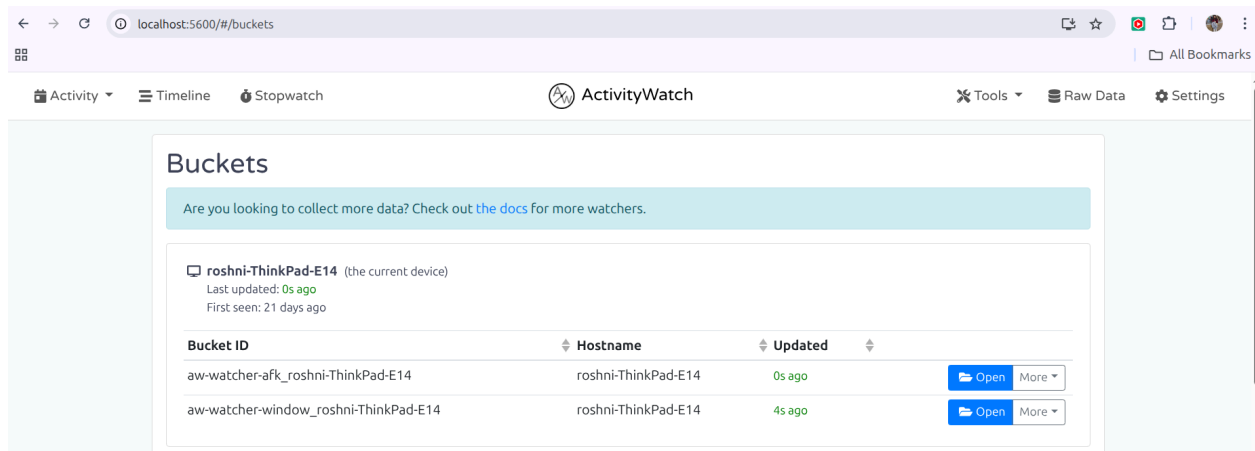
print(f"✅ Activity summary generated successfully.")

for laptop_id in result.keys():
    send_to_google_script(laptop_id, category_summary.get(laptop_id, {}),
result.get(laptop_id, []))

```

Here:

```
laptops = [  
    { "id": "PG026RXY", "hostname": "roshni-ThinkPad-E14", "bucket":  
      "aw-watcher-window_roshni-ThinkPad-E14", "host": "localhost"  
    }, ]
```



Instead Of hostname put you laptop id and you bucket ID (2nd one) and in place of ID:

a.) Run this command:

```
sudo dmidecode -s system-serial-number /  
(for windows wmic bios get serialnumber  
and copy you serial number.
```

For checking the data on terminal

```
python activity_tracker.py
```

🏆 Step a: In terminal run command(For hide the folder):
`mv ~/Downloads/activitywatch ~/.activitywatch`

🏆 Step b: Add ActivityWatch to Startup

1. Click on Add

2. In the pop-up:

- Name: **ActivityWatch**

Run on terminal:

```
find ~/.activitywatch -type f -name "aw-qt"
```

- Command: something like
`/home/sama/.activitywatch/activitywatch/activitywatch/aw-qt`
- Comment: (Leave empty or write **Auto start ActivityWatch**)

3. Click Add and then close.

 **Done! Now every time you log in, ActivityWatch will start automatically.**

1.) In terminal run command:

```
mv ~/activity_tracker.py ~/.activity_tracker.py
```

2.) Now in terminal run command:

```
crontab -e
```

3.) Paste this : `0 11 * * * /usr/bin/python3`

```
/home/sama/.activity_tracker.py >> /home/sama/activity_log.txt 2>&1
```

```
@reboot /usr/bin/python3 /home/sama/.activity_tracker.py >>  
/home/sama/activity_log.txt 2>&1
```

4.) Press `Ctrl+0` then `Enter` and `Ctrl+X`

(make it executable)

```
chmod +x /home/username/scripts/activity_script.py
```

5.) (This command for just testing)

```
/usr/bin/python3 /home/sama/.activity_tracker.py
```